Verification as a Library: Symbolic Programs and Compositional Proofs in SBV

Experience Report

LEVENT ERKOK

A verification library must provide a language for models, control over solver interaction, and support for proofs that exceed push-button automation. SBV supplies these facilities in Haskell: typed symbolic values describe computations, query programs orchestrate solver sessions, and proof programs combine induction and equational reasoning with SMT discharge of local obligations. This report argues that effective library embedding requires composition at all three levels. We explain the mechanisms that enable it: deferred graph construction with two forms of sharing, incremental discovery of declarations and definitions, and proof decomposition with explicit hypotheses and termination obligations. Executable examples cover machine arithmetic, model enumeration, symbolic datatypes, accumulator reversal, and semantics preservation for constant folding. C generation supplies a second consumer of the symbolic representation and a concrete validation boundary. The account identifies what Haskell contributes to solver-aided programming, which semantic choices must remain visible, and how to test the contracts connecting expressions, sessions, proofs, and generated execution.

Additional Key Words and Phrases: Haskell, symbolic programming, SMT, embedded languages, induction, verification

1 INTRODUCTION

A verification library should let a programmer begin with a small question and retain the same modeling vocabulary as the question becomes harder. Checking an arithmetic identity, searching for a counterexample, adapting a query to a model, and proving a recursive transformation require different reasoning strategies. They need not require different representations of the program being studied. SBV [Erkok 2026] explores this design space in Haskell: computations over typed symbolic values can be used in automated queries, interactive solver programs, and structured proofs.

A small example shows both the appeal and the semantic precision of this approach:

```
prove $ \x -> abs (x :: SInteger) .>= 0
-- Q.E.D.

prove $ \x -> abs (x :: SInt8) .>= 0
-- Counterexample: x = -128
```

The first theorem concerns mathematical integers. The second concerns signed eight-bit arithmetic, where the positive value 128 is unrepresentable and `abs (-128)` is -128. The type changes the theorem, while the property retains its structure. This is more than convenient notation: symbolic types select the domain in which the solver must interpret a program.

The central challenge is to preserve this convenience beyond isolated formulas. Haskell evaluates the program that builds a symbolic computation; it does not automatically turn arbitrary Haskell execution into logic. A solver session has state, declaration order, and model lifetimes that are absent from a pure arithmetic expression. A recursive theorem needs an induction argument that a solver may not discover. SBV addresses these problems through a layered interface: symbolic expressions describe what is to be reasoned about, query programs describe how to interact with the solver, and proof programs describe how a theorem reduces to smaller obligations.

The thesis is that *verification as a library depends on composition at all three levels*. Expressions supply a modeling vocabulary; programmable queries embed reasoning in algorithms; structured

Table 1. Ways to use a common symbolic model. Each interface adds a distinct kind of control or result.

Interface	Programmer supplies	Result or capability
`prove`	Property and assumptions	Proof result or counterexample
`sat`, `allSat`	Constraints on unknowns	Models satisfying the constraints
`optimize`	Constraints and objectives	Models optimized under a chosen policy
`query`	A solver interaction algorithm	Adaptive checks and model inspection
TP	Lemmas, induction, and calculations	Checked steps and dependencies
C generation	An executable symbolic computation	Executable C code

proofs let users supply mathematical arguments while automating their local steps. A shared symbolic representation connects these uses and supports compilation of an executable subset to C.

The report makes three contributions:

(1) An architectural explanation of how SBV combines a typed, largely pure expression interface with deferred construction of a shared graph. We separate host evaluation, logical meaning, and symbolic-run identity, showing why each requires its own mechanism (Sections 2–3).
(2) An account of compositional solver and proof interfaces: dependency synchronization allows new expressions inside live queries, while explicit induction and calculations turn recursive correctness claims into local SMT obligations (Sections 4–5).
(3) Executable case studies and a validation method connecting specification, proof, and generated execution. These identify distinct obligations for the embedding, solver, proof layer, and C backend, and yield concrete design lessons for other embedded reasoning systems (Sections 6–9).

The contribution is the design and analysis of this composition, supported by implementation mechanisms and reproducible examples. The underlying solver algorithms, observable sharing, and induction principles are established techniques. We concentrate on how a library makes them work together.

2 A MODELING LANGUAGE WITH SEVERAL USES

2.1 Types choose the mathematical problem

SBV supplies symbolic booleans, bounded signed and unsigned words, mathematical integers and reals, IEEE floating-point values, strings, sequences, arrays, sets, and algebraic datatypes. These are modeling choices, not interchangeable representations of one universal numeric or collection type. Bit-vector addition wraps; integer addition does not. Floating-point equality is not reflexive on NaNs. An SMT array describes a total mapping with read and write operations; it does not inherit the allocation and bounds behavior of a Haskell array.

There is a similar distinction between `[SInteger]` and `SList Integer`. The former is a Haskell list whose spine is manipulated by Haskell and whose elements may be symbolic. Its concrete length can guide construction of a fixed-size formula. The latter is a symbolic finite sequence: its length and contents can both be unknown, and operations on it are represented in logic. Choosing between them determines whether a traversal executes while building the problem or remains part of the problem to be solved.

Table 1 summarizes the uses of this vocabulary. Solver configurations select a backend, including Z3 [de Moura and Bjørner 2008], and SMT-LIB provides the principal interchange language [Barrett et al. 2017]. The available theories and reasoning strength depend on that backend.

2.2 One implementation, an independent specification

Consider unsigned-byte addition that saturates at 255. The implementation can detect overflow before adding, while the specification can use unbounded integers:

```
satAdd :: SWord8 -> SWord8 -> SWord8
satAdd x y = ite (x .> 255 - y) 255 (x + y)

satAddSpec :: SWord8 -> SWord8 -> SBool
satAddSpec x y = sFromIntegral (satAdd x y) .==
                 ite (total .> 255) 255 total
  where total = sFromIntegral x + sFromIntegral y :: SInteger
```

The guard is safe because `255 - y` is representable for every byte `y`. On the other branch, `x + y` is known not to overflow. In the specification, conversion occurs *before* addition. Converting an already wrapped sum to an integer would preserve the wrong value and could make the specification repeat the implementation's error.

Writing `prove satAddSpec` asks for the absence of a counterexample over all 65,536 input pairs. More generally, for assumptions A and a property P, a universal proof query checks satisfiability of

$$A(\vec{x}) \wedge \neg P(\vec{x}).$$

Unsatisfiability establishes the property under the assumptions; a satisfying model supplies inputs that refute it. An inconclusive response establishes neither. Satisfiability queries use the same modeling language to search for values instead of refuting a universal claim. Optimization adds objectives and a policy for combining them, such as lexicographic order or Pareto optimization. Thus the operations on symbolic values describe the problem independently of the reasoning task chosen for it.

The distinction between domains also produces a useful negative control. The tempting implementation

```
wrongAdd :: SWord8 -> SWord8 -> SWord8
wrongAdd x y = ite (x + y .> 255) 255 (x + y)
```

never takes its saturation branch: a byte sum cannot exceed 255. The companion asks for a counterexample to this implementation and checks the returned pair using ordinary integer arithmetic. A successful proof of the corrected program and a concrete refutation of this plausible error exercise both outcomes of the verification interface.

2.3 What Haskell contributes

For literal arguments, symbolic operations can evaluate immediately: `satAdd 250 10` yields the literal byte 255. For unknown arguments, the same definition constructs a graph. Haskell functions, higher-order combinators, modules, and type classes organize this construction. A function can generate a family of circuits or constraints without those construction abstractions appearing in the resulting logic.

The embedding uses ordinary overloaded operators where their types permit it. `Num` supports symbolic addition and literals. Haskell's `Eq`, however, returns `Bool`; symbolic equality must return `SBool`, hence `.==`. Ordinary `if` chooses a branch during Haskell evaluation; `ite` represents a choice whose condition may remain unknown. This explicit boundary lets a library retain ordinary Haskell execution while constructing logical computations. It also means that verifying a

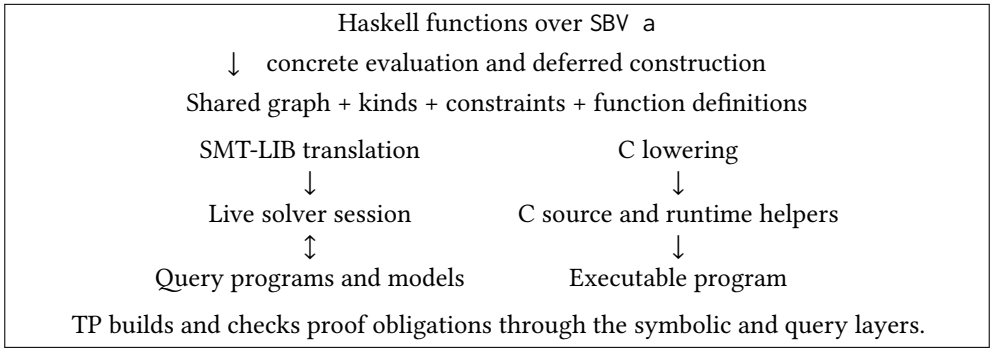


Fig. 1. One representation supports several reasoning and execution workflows. The shared front end enables reuse; each consumer retains its own correctness obligations.

symbolic model is a claim about that model's semantics, rather than an automatic verification of arbitrary Haskell source.

3 A PURE INTERFACE OVER A SHARED SYMBOLIC GRAPH

3.1 Deferred construction connects expressions to state

The central type is `SBV a`; for example, `SWord8` abbreviates `SBV Word8`. Its internal representation has the following shape, omitting fields irrelevant to the discussion:

```
newtype SBV a = SBV { unSBV :: SVal }
data SVal = SVal Kind (Either CV (Cached SV))
data SV = SV Kind NodeId
data SBVExpr = SBVApp Op [SV]
newtype Cached a = Cached (State -> IO a)
```

The phantom parameter carries the public Haskell type. A run-time `Kind` records the logical sort, and `SymVal` connects Haskell types with symbolic kinds and literal representations. A value contains either a concrete payload or a deferred computation that produces a graph node in a particular symbolic state. This indirection explains how expressions such as `x + y` can have an ordinary pure-looking type even though allocating and reusing their graph nodes requires state.

Operations fold concrete arguments when possible and otherwise construct applications. Simplifications must respect the operand kind: replacing `x .== x` with true is valid for integers but not for floating-point values that may be NaNs. When a deferred value is demanded in a symbolic run, its operands are materialized and its application is recorded. The state accumulates inputs, constants, assignments, constraints, sorts, and function definitions. `SymbolicT` exposes the state through a reader transformer; mutable tables implement the graph and caches. Clients can integrate construction with their own effects without making every arithmetic operation monadic.

This organization trades a fully type-indexed internal syntax for a compact, uniform representation. Public types reject many invalid combinations; run-time checks must still ensure agreement between kinds, payloads, and operation signatures. The type discipline and the representation checks are complementary parts of the design.

3.2 Sharing construction and sharing expressions

Two kinds of sharing are needed. First, forcing the same deferred computation repeatedly should reuse its result within a run. SBV memoizes these computations using stable names. Second, separately constructed applications of the same operator to the same symbolic nodes should reuse a graph node. The expression table implements this second form of sharing, with normalization of selected argument orders and a check that the reused result has the requested kind.

The distinction matters for a generator that repeatedly combines a term with itself. If reification follows both occurrences afresh at every level, a linearly described family can expand into an exponentially large tree. Caching the deferred construction avoids revisiting shared host computations; interning applications also finds duplication introduced by different construction paths. The former concerns the work of building the graph, the latter the graph that is built. Neither equates arbitrary mathematically equivalent expressions.

Observable sharing is an established technique for embedded languages [Gill 2009]. Its role here is to reconcile ordinary expression syntax with an explicitly shared intermediate representation. A global cache of graph nodes would be inappropriate: a node identifier is meaningful in the run that allocated it. SBV therefore associates memoized construction with the symbolic state rather than treating a Haskell expression as a globally valid solver object.

This also explains a limit of the public types. Two values of type `SInteger` need not belong to the same run. Nodes carry context information, and construction checks for common context mismatches. A concrete `Integer` extracted from a model can safely be made into a literal in another run; moving a run-specific symbolic node does not have the same meaning. A type that identifies a logical sort does not, by itself, identify the lifetime of the external resource interpreting that sort.

3.3 Recursion must occur at the intended level

A recursive Haskell function under a symbolic `ite` can keep unfolding while constructing branches. The finiteness of an input type does not ensure that construction terminates: the stopping condition may remain symbolic. There are two useful ways to control this. A concrete bound can drive a finite host-language unfolding, appropriate for bounded models and fixed-size circuits. Alternatively, `smtFunction` records a named logical definition and represents recursive calls as calls, preserving recursion in the target logic.

These choices answer different questions. A bound restricts the generated model. A recursive definition describes behavior beyond a fixed unfolding, but requires well-founded definitions and may require induction to prove its properties. Moving recursion into logic is therefore both a representational choice and a change in the reasoning obligations. Section 5 shows how SBV lets the programmer supply that missing proof structure.

4 PROGRAMMING THE SOLVER

4.1 Models become inputs to the next computation

A fixed invocation of `prove` or `sat` is insufficient when the next question depends on the previous answer. SBV's query interface gives Haskell control over a live solver session. Consider enumerating the integer roots of $x^2 = 9$:

```
roots :: IO [Integer]
roots = runSMT $ do
  x <- sInteger "x"
  constrain $ x * x .== 9
  query $ let loop = do
                result <- checkSat
                case result of
                  Sat -> do v <- getValue x
                            constrain $ x ./= literal v
                            (v :) <$> loop
                  Unsat -> pure []
                  _ -> io $ fail "Root enumeration was inconclusive"
          in loop
```

Each satisfying model supplies a concrete root. The program constructs a new constraint excluding that root, and asks again in the same session. The solver can return the roots in either order. Unsatisfiability after both exclusions establishes that enumeration is complete; an unknown result is an incomplete computation and takes a separate branch.

The loop illustrates a general pattern: solve, inspect, refine. Enumeration, application-specific search, and counterexample-guided refinement can all use Haskell control flow around symbolic constraints. The host branch on `Sat` is concrete; the new constraint remains symbolic. Keeping those two forms of control distinct makes the solver usable as a component within a larger algorithm. Assertion-stack operations support temporary hypotheses, while named constraints and model queries support diagnosis and inspection.

Enumeration also needs a stopping argument. Here the constraint has only two solutions. Over an unbounded domain, a loop that excludes one model at a time can run indefinitely even when each solver call terminates. The interface supplies the mechanism; the client algorithm supplies its progress and stopping conditions.

4.2 Incrementality is dependency management

New constraints can introduce new constants, sorts, expressions, and recursive functions after the solver has started. An incremental API must therefore do more than send another assertion. It must ensure that every referenced object has a declaration or definition available in the live session.

The implementation maintains both the accumulated symbolic state and an incremental component containing newly discovered objects. A query operation that materializes a value runs in a fresh incremental context, records the objects it introduces, and synchronizes them before subsequent use. The synchronization path processes pending definition checks and emits the new sorts, inputs, constants, definitions, assignments, and constraints in a form the solver can accept. It also distinguishes uninterpreted function declarations from functions whose definitions are now available.

A useful specification of this behavior is the following dependency invariant. Let D be the objects available in the current solver context and $\mathrm{deps}(e)$ the declarations and definitions needed by a new expression e. Before a command uses e, synchronization must establish

$$\mathrm{deps}(e) \subseteq D.$$

Dependencies include those required by other dependencies. The implementation realizes this through kind registration, function discovery, and incremental translation; the invariant summarizes the obligation rather than claiming a formal proof of the protocol.

This is what makes the expression interface compose with the query interface. A helper function should carry the dependencies of the terms it constructs, whether it is called during initial setup or after a model has been inspected. Requiring an unrelated earlier use merely to register a type would couple clients to translation order. Genuine solver restrictions, such as options that must precede session initialization, remain explicit protocol constraints.

4.3 Symbolic datatypes make dependencies structural

Template Haskell's `mkSymbolic` derives symbolic support for user datatypes, and `sCase` supplies symbolic pattern matching. Constructors, recognizers, and selectors connect the host declaration to the corresponding logical structure. Ordinary Haskell pattern matching still runs in the host; symbolic matching builds the tests and branch values needed for an unknown constructor.

The companion exercises a deliberately awkward entry point:

```
newtype Box a = Box a deriving (Eq, Ord, Show)
mkSymbolic [''Box]

lateDatatype :: IO CheckSatResult
lateDatatype = runSMT $ query $ do
  x <- freshVar @(Box Word8) "x"
  y <- freshVar @(Box Word8) "y"
  constrain $ x ./= y
  checkSat
```

Both `Box Word8` values are first introduced inside the query. Their inequality is satisfiable, but only if the datatype has been registered before its variables and operations reach the solver. For larger datatypes, a required sort can occur under a tuple, sequence, set, array, or type argument. Recursive definitions can introduce cycles. Dependency discovery must therefore follow type structure rather than inspecting only the immediately visible fields.

This yields a practical testing method: vary both *what* is introduced and *where* it is first introduced. SBV's datatype-registration tests cross nested representations with setup-time and query-time variable creation. The relevant correctness property is not merely that an operation works once its dependencies happen to exist; it is that composing the operation into a valid client program brings those dependencies with it.

5 FROM SOLVER AUTOMATION TO COMPOSITIONAL PROOFS

5.1 The programmer supplies the induction structure

SMT automation is effective for many local arithmetic and datatype facts. Recursive correctness claims often need a stronger invariant, induction, case analysis, or intermediate equalities. SBV's TP layer expresses this structure as a Haskell proof program, generating obligations for the same symbolic and query infrastructure.

Accumulator-based reversal is a compact example. The desired top-level result is reversal into an empty accumulator, but the recursive call changes the accumulator. The useful theorem is therefore

$$P(xs) \equiv \forall acc.\ \mathrm{revAcc}(acc, xs) = \mathrm{reverse}(xs) \mathbin{++} acc.$$

The companion expresses the recursive function and its proof together:

```
revAcc :: SList Integer -> SList Integer -> SList Integer
revAcc = smtFunction "paper.revAcc" $ \acc xs ->
  [sCase| xs of
    []     -> acc
    x : xt -> revAcc (x .: acc) xt
  |]

revAccCorrect :: TP (Proof (Forall "xs" [Integer]
                         -> Forall "acc" [Integer] -> SBool))
revAccCorrect = induct "revAccCorrect"
  (\(Forall xs) (Forall acc) ->
      revAcc acc xs .== SL.reverse xs SL.++ acc) $
  \ih (x, xs) acc -> [] |- revAcc acc (x .: xs)
                       =: revAcc (x .: acc) xs
                       ?? ih
                       =: SL.reverse xs SL.++ (x .: acc)
                       =: (SL.reverse xs SL.++ [x]) SL.++ acc
                       =: SL.reverse (x .: xs) SL.++ acc
                       =: qed
```

Here `Forall` names quantified parameters, `induct` provides list induction, `|-` introduces the calculation under assumptions, `=:` connects successive expressions, and `??` supplies a proof hint. The named recursive function remains a logical definition; Haskell does not unfold it for all possible input lengths.

The quantifier placement is the mathematical core of the proof. At the inductive step the hypothesis must be available for *every* accumulator, so it can be instantiated with `x .: acc`. A statement specialized to the empty accumulator would not justify that recursive call. The base case reduces to $acc = [] \mathbin{+\!\!+} acc$. For the step, the supplied calculation exposes exactly the transformations needed:

$$\begin{aligned}
\mathrm{revAcc}(acc, x : xs) &= \mathrm{revAcc}(x : acc, xs) && \text{definition}\\
&= \mathrm{reverse}(xs) \mathbin{+\!\!+} (x : acc) && \text{induction hypothesis}\\
&= (\mathrm{reverse}(xs) \mathbin{+\!\!+} [x]) \mathbin{+\!\!+} acc && \text{sequence laws}\\
&= \mathrm{reverse}(x : xs) \mathbin{+\!\!+} acc && \text{definition.}
\end{aligned}$$

The user chooses the theorem and the intermediate expressions; the solver checks the local consequences. This division of labor is the reason a small proof language can be useful without implementing a general proof-search procedure for induction.

5.2 Calculations are programs that generate obligations

A calculation $e_0 = e_1 = \cdots = e_n$ under assumptions A and applicable lemmas L is checked through obligations of the form

$$A \wedge L \wedge \neg(e_i = e_{i+1}).$$

In ordinary checked execution, the proof-step machinery accepts unsatisfiability and rejects satisfiable or inconclusive responses. Induction and case-split combinators additionally organize the conditions under which those steps suffice for the claimed result. Proof objects record propositions and dependencies; reusable lemmas let later proofs work at the level of a previously established interface rather than repeatedly exposing definitions.

For strong induction on a measure m, the available hypothesis has the shape

$$\big(\forall y.\ m(y) < m(x) \Rightarrow P(y)\big) \Rightarrow P(x),$$

with a well-founded measure domain and its accompanying obligations. Recursive function definitions also need justified termination. These are related but distinct requirements: an induction measure justifies use of a hypothesis, while a function's termination argument justifies its recursive definition. SBV supplies measure checks and records functions whose termination checking was explicitly bypassed. Supplying an axiom or bypassing a check changes the dependencies of the result and must be understood as such.

5.3 Case study: semantics-preserving constant folding

The library's constant-folding development shows how this approach scales in *proof structure*. Its expression language has seven constructors: variables, constants, squaring, increment, addition, multiplication, and local bindings. An environment-based interpreter gives the language meaning. The transformation recursively folds children, then simplifies their parent; for example, constant arithmetic becomes a constant and multiplication by one disappears. Its theorem is

$$\forall e, \rho. \quad \text{interpInEnv}(\rho, \text{cfold}(e)) = \text{interpInEnv}(\rho, e).$$

This quantifies over all finite represented expressions and environments, not just expressions up to an externally selected test depth.

The interesting case is local binding. Simplifying $\text{Let}(n, \text{Con}(v), b)$ substitutes v for n in b. A substitution must respect shadowing: in a nested binding of the same name, it still processes the binding's right-hand side, but leaves the bound body alone. The central supporting equation is

$$\text{interpInEnv}(\rho, \text{subst}(n, v, b)) = \text{interpInEnv}((n, v) : \rho, b).$$

Proving this requires facts about environment lookup and how bindings interact. These facts are reusable semantic lemmas, rather than additional clauses in one large solver query.

The proof decomposes into three layers. Environment and substitution lemmas establish the meaning of replacing a variable. A simplification theorem shows that each local rewrite preserves interpretation. Finally, strong induction on an expression-size measure combines the recursive hypotheses with that simplification theorem. The main proof splits over all seven constructors; the binding case uses the induction hypothesis at an updated environment. As with the accumulator in reversal, generalizing over the environment is necessary because recursive reasoning changes it.

This case study demonstrates the practical contribution of the proof layer: symbolic datatypes describe syntax, named functions describe semantics and transformations, and proof combinators organize the correctness argument in the same host language. It also locates the remaining intellectual work. SBV can check the substitution lemma, but the programmer must identify that lemma and choose a sufficiently general statement. Automation is most useful when the proof program presents the right interface between subproblems.

The domain of these theorems remains explicit. `SList` describes finite logical sequences, not arbitrary lazy Haskell lists including bottoms and infinite values. The constant-folding theorem concerns its symbolic interpreter. Connecting that interpreter to a separately implemented language runtime would require an additional correspondence argument.

6 FROM SYMBOLIC COMPUTATION TO EXECUTABLE C

A shared symbolic representation can serve execution as well as reasoning. The saturating adder is compiled by specifying its input and output interface:

```
generateAdder :: FilePath -> IO ()
generateAdder dir = compileToC (Just dir) "sat_add" $ do
  cgOverwriteFiles True
  cgGenerateDriver False
  x <- cgInput "x"
  y <- cgInput "y"
  cgReturn $ satAdd x y
```

The generated function runs without a solver. Reusing `satAdd` keeps the verified symbolic algorithm and the input to code generation aligned, while the backend remains responsible for implementing each operation faithfully. This is useful even without a verified compiler: it removes a manual rewrite of the algorithm and makes the remaining translation boundary explicit.

6.1 A logical term needs an operational implementation

The difficult part is not printing an operator in C syntax. Logical values may need allocated storage, initialization, copying, and runtime support. A lowering result therefore contains four components: the expression producing the value, declarations for storage, setup statements that must precede evaluation, and the runtime capabilities it requires. The distinction between declarations and setup is essential for conditionals. Storage declarations can sometimes be hoisted; setup for a branch must execute where that value is demanded. Eagerly moving both branches' setup before a conditional can introduce operations that the selected branch never needed.

Numeric representations impose further obligations. Bit-vector operations need specified widths and signedness instead of accidental dependence on C integer promotion or signed overflow. Exact integers and rational-valued reals can use GMP-backed representations; floating-point operations may need rounding support and LibBF. The mathematical-real domain of SMT is larger than exact rational values, so an executable real representation cannot be treated as an unrestricted implementation of every solver-level real term. Similarly, quantifiers are reasoning constructs rather than executable enumeration instructions.

These examples show why the shared graph is a semantic interface, not a promise that every consumer implements its whole vocabulary. A backend must either provide the required interpretation under stated assumptions or reject the unsupported construct. Silent approximation would change the meaning of a previously verified property.

6.2 Validate the translation boundary independently

There are two separate correctness statements for the adder:

$$\forall x, y \in \{0, \ldots, 255\}. \quad \mathrm{satAdd}_{\mathrm{SMT}}(x, y) = \min(255, x + y),$$
$$\forall x, y \in \{0, \ldots, 255\}. \quad \mathrm{satAdd}_{\mathrm{C}}(x, y) = \min(255, x + y).$$

The proof query establishes the first, subject to the embedding and solver. The companion checks the second by compiling the generated function and executing all input pairs against an independent wider-arithmetic oracle. Neither statement alone establishes the other. Together they give a complete finite-domain comparison for this function in the recorded compilation environment, without turning that result into a claim about the entire backend.

This suggests a general testing discipline for a representation with several consumers. Compare concrete evaluation and solver interpretation, then compare generated execution with an independent mathematical expectation. Selected C regressions additionally vary optimization settings and enable sanitizers. Independent oracles matter because a shared implementation error can make two consumers agree with each other while both disagree with the intended semantics.

Table 2. Executable evidence for the report's claims. The checks exercise specific semantic and architectural obligations.

Task	Required outcome
Integer absolute value	Universal nonnegativity theorem succeeds
Signed-byte absolute value	Counterexample is −128
Floating-point equality	Reflexivity has a NaN counterexample
Saturating addition	Implementation satisfies the integer specification
Incorrect overflow guard	Returned counterexample fails an integer oracle
Incremental roots	Exactly −3 and 3, then unsatisfiability
Query-created datatype	Distinct byte boxes are satisfiable
Accumulator reversal	Generalized inductive proof completes
Constant folding	Existing semantics-preservation development completes
Generated C addition	All 65,536 pairs agree with an independent oracle

7 EVIDENCE AND REPRODUCIBILITY

The companion contains executable Haskell examples, an independent C oracle, and reproduction scripts. The five main listings are extracted from the checked source. All ten tasks in Table 2 completed successfully: eight small Haskell checks, the constant-folding proof, and exhaustive C execution. For the incorrect adder, the counterexample was 223 and 124: their integer sum is 347, their byte sum is 91, and the specified saturated result is 255. The execution record supplies the source revision and tool versions.

The checks inspect model values rather than rely on root order or printed transcripts. Timeouts are incomplete checks, not logical answers. The constant-folding task runs the existing development with CVC5. C validation uses optimization, strict warnings, and undefined-behavior sanitization.

The repository adds complementary regression techniques: QuickCheck properties compare concrete symbolic literals with solver interpretation; datatype tests vary nesting and phase of first use; compile-failure tests check diagnostics for symbolic pattern matching; and generated-code tests exercise runtime semantics. These target different obligations that no single arithmetic proof can establish. The complete regression suite is outside the companion run.

These selected examples support the architectural claims, not a comparative performance or usability evaluation. They do not measure adoption or development effort. In particular, the unbounded inductive statements do not establish scalability for arbitrary recursive theorems: quantifiers, nonlinear arithmetic, and recursive definitions remain sources of incomplete or expensive reasoning.

8 RELATED WORK

Properties and symbolic execution in a host language. QuickCheck [Claessen and Hughes 2000] makes executable properties part of ordinary Haskell development, searching for counterexamples by generated testing. SBV uses a related library style but represents supported operations as logical formulas. The distinction lies in the evidence obtained: a successful finite test campaign and an unsatisfiability result establish different claims. SBV's own tests combine these techniques.

Smten [Uhler and Dave 2013] translates high-level symbolic computations into solver queries, addressing the difficulty of producing useful encodings from program structure. Rosette [Torlak

and Bodík 2014] supplies a symbolic virtual machine for solver-aided host languages, using concrete execution to eliminate host constructs outside its symbolic core. SBV likewise depends on host execution to construct a logical computation, but exposes symbolic types and control operators directly through a Haskell library. The visible host/symbolic boundary is a central interface choice in the present account.

Grisette [Lu and Bodík 2023] is a particularly close comparison: symbolic compilation is also a typed Haskell library. Its purely functional representation and ordered-guards technique address symbolic state merging and constraint size. SBV's focus here is a complementary architectural problem: connecting shared expression construction, live solver sessions, explicit inductive proofs, and executable code through one representation. Library embedding alone is therefore not a novelty claim; the contribution is the analysis of how these interfaces compose and which obligations their composition creates.

Representation and proof structure. Gill [2009] explains type-safe observable sharing for Haskell embedded languages. SBV uses stable names to memoize deferred construction and expression interning to reuse graph applications. The distinction between these two mechanisms, and their dependence on symbolic-run state, is the relevant design point here.

Refinement reflection [Vazou et al. 2018] connects user-defined functions with SMT reasoning through refinement types, supporting equational proofs and inductive reasoning in Liquid Haskell. SBV's TP instead makes the proof a program that constructs and executes solver obligations over explicitly symbolic definitions. Both give users a way to supply structure that SMT automation needs. They differ in where specifications and proofs live: in a refinement-type verification framework versus an embedded proof API run by Haskell. This distinction is more informative for choosing an approach than comparing the surface similarity of their equational notation.

9 DESIGN LESSONS AND TRUST

Give each level of composition its own contract. Expression composition requires compatible sorts and graph contexts; query composition requires available dependencies; proof composition requires valid hypotheses and induction. A well-typed term can still reference the wrong run, and a correctly declared recursive function can still need a stronger theorem. Testing these contracts separately exposes failures that end-to-end examples alone can miss.

Expose semantic choices and make reasoning programmable. Haskell supplies abstraction and construction, while symbolic types distinguish bounded words from integers and host lists from logical sequences. Query procedures supply search strategies; proof programs supply generalized invariants and semantic lemmas. These reusable units let clients organize reasoning around the solver's strengths while retaining one modeling vocabulary.

Validate shared representations with independent expectations. A common graph reduces duplicate modeling but concentrates risk in shared constructors and simplifications. The adder's integer specification avoids repeating its bounded arithmetic; its execution oracle avoids reusing the symbolic implementation. Both check intended mathematics, rather than agreement with the system's own output.

The trusted base includes the Haskell implementation and runtime, symbolic encoding, solver, and TP's obligation generation. User axioms and unchecked termination assumptions add dependencies when used. C generation additionally relies on its lowering, runtime support, and toolchain. TP checks solver outcomes and records dependencies; its ordinary workflow does not replay certificates in an independent small proof kernel. Verified translation and certificate checking would establish stronger guarantees than the validation reported here.

10 CONCLUSION

SBV connects typed expressions, live solver interaction, and structured proofs through a shared symbolic representation. Deferred construction preserves expression sharing; incremental synchronization carries dependencies into solver sessions; proof programs reduce recursive claims to local obligations. C generation adds execution with a separate translation contract. The examples locate the programmer's contribution precisely: selecting the semantic domain, choosing a general enough invariant, and validating the connection to execution. For embedded reasoning systems, composition therefore requires contracts for values, sessions, and proofs as well as a convenient syntax for formulas.

REFERENCES

Clark Barrett, Pascal Fontaine, and Cesare Tinelli. 2017. *The SMT-LIB Standard: Version 2.6.* Technical Report. Department of Computer Science, The University of Iowa. https://smt-lib.org/language.shtml The language site also provides subsequent revisions.

Koen Claessen and John Hughes. 2000. QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs. In *Proceedings of the ACM SIGPLAN International Conference on Functional Programming*. ACM, New York, NY, USA, 268–279. https://research.chalmers.se/en/publication/237427

Leonardo de Moura and Nikolaj Bjørner. 2008. Z3: An Efficient SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems*. Springer, Berlin, Heidelberg, 337–340. https://doi.org/10.1007/978-3-540-78800-3_24

Levent Erkok. 2026. SBV: SMT Based Verification in Haskell. Software and executable documentation. https://github.com/LeventErkok/sbv The companion execution record identifies the source revision used for validation.

Andy Gill. 2009. Type-Safe Observable Sharing in Haskell. In *Proceedings of the Second ACM SIGPLAN Symposium on Haskell*. ACM, New York, NY, USA, 117–128. https://doi.org/10.1145/1596638.1596653

Sirui Lu and Rastislav Bodík. 2023. Grisette: Symbolic Compilation as a Functional Programming Library. *Proceedings of the ACM on Programming Languages* 7, POPL, Article 16 (2023), 33 pages. https://doi.org/10.1145/3571209

Emina Torlak and Rastislav Bodík. 2014. A Lightweight Symbolic Virtual Machine for Solver-Aided Host Languages. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, New York, NY, USA, 530–541. https://doi.org/10.1145/2594291.2594340

Richard Uhler and Nirav Dave. 2013. Smten: Automatic Translation of High-Level Symbolic Computations into SMT Queries. In *Computer Aided Verification*. Springer, Berlin, Heidelberg, 678–683. https://doi.org/10.1007/978-3-642-39799-8_45

Niki Vazou, Anish Tondwalkar, Vikraman Choudhury, Ryan G. Scott, Ryan R. Newton, Philip Wadler, and Ranjit Jhala. 2018. Refinement Reflection: Complete Verification with SMT. *Proceedings of the ACM on Programming Languages* 2, POPL, Article 53 (2018), 31 pages. https://doi.org/10.1145/3158141